



WebGL → WebGPU

A piece-by-piece migration guide for the Three.js WebGPURenderer

Three.js is an easy-to-use, lightweight, cross-browser 3D library. The WebGPURenderer targets WebGPU — a newer, high-performance graphics and compute API designed for today's hardware. The migration is straightforward: swap the import, swap the renderer, then audit shaders and effects.

You can use the WebGPURenderer inside Three.js today.

► since r167

◆ REALITY CHECK

The WebGPURenderer is still experimental — expect some bumps. A few features won't port 1-to-1 (custom shaders, EffectComposer). The best move is to migrate piece by piece, not all at once.

The Migration Path

1 Update Imports

Point your import at the WebGPU entry point.

```
import * as THREE from 'three' → 'three/webgpu';
```

- update your import maps (see the [threejs.org](#) examples)

2 Change Renderers

Swap the constructor. Your usual setSize and appendChild setup stays the same.

```
// was new THREE.WebGLRenderer();  
const renderer = new THREE.WebGPURenderer();
```

3 Handle Async Behavior

WebGPU initializes asynchronously. Use setAnimationLoop() — it waits until the renderer is ready. Lose requestAnimationFrame.

```
renderer.setAnimationLoop(animate);  
  
// init first:  
await renderer.init();
```

- Remove your old requestAnimationFrame()
- Call await renderer.init() first

4 Automatic Fallback

Without WebGPU support, it falls back to WebGL2 automatically. Confirm it still runs in those browsers.

```
// force-test the fallback path  
new THREE.WebGPURenderer({ forceWebGL: true });
```

- [threejs.org](#) has excellent TSL & node examples

5 Materials Audit

Move custom shaders to Node Materials + TSL (shader logic in JavaScript). Swap to a basic material first, then port effects one at a time.

◆ UNSUPPORTED:

ShaderMaterial onBeforeCompile RawShaderMaterial

6 EffectComposer → TSL Nodes

EffectComposer isn't supported. Post-processing becomes node compositions written in TSL — most common effects already have WebGPU- friendly node versions, and the new RenderPipeline is more performant.

- WebGPU-onLy: **SSGI** • **SSS** • **better Depth of Field**

• Updating an old project?

Bump the library in increments of 10 releases — deprecation warnings last 10 releases, so stepping keeps API changes manageable.

The WebGPURenderer is the future.

If something is broken or missing, that's just the renderer maturing — improvements land every month, and all the big new-feature momentum is around it. Hit a wall? Keep an eye on updates, and file an issue.

improvements
ship monthly